

Министерство образования и науки РФ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Омский государственный технический университет»

Кафедра «Информатика и вычислительная техника»

Отчёт по лабораторной работе № 1
по дисциплине
«Проектирование и тестирование программного обеспечения»

Выполнил

Студент гр. ПИ-221

Нурсалимов Д.Е.

18.05.24

(подп., дата)



Проверил

Ст. пр. Звонов А. О 18.05.24

(подп., дата)



Омск 2024

Задача 1.2 (Сапёр)

PC/UVa IDs: 110102/10189

Как известно, цель игры состоит в том, чтобы найти расположение всех мин на поле размером $M \times N$. Игра показывает вам число в клеточке, которое говорит вам, сколько мин находится в соседних с этой клеточках. Каждая клеточка имеет максимум восемь соседних. Поле 4×4 слева содержит две мины, каждая из которых представлена символом «*». Если мы добавим к этому полю числа-подсказки по принципу, описанному выше, то получим поле, изображенное справа:

* . . .	*100
. . . .	2210
. * . .	1*10
. . . .	1110

Входные данные

Входные данные будут состоять из произвольного числа полей. Первая строка каждого поля содержит два целых числа n и m ($0 < n, m \leq 100$), которые соответствуют числу строк и столбцов поля соответственно. Каждая из последующих n строк содержит ровно m символов, представляя собой поле.

Безопасные клеточки обозначаются «.» , мины обозначаются «*», и то и другое без кавычек. Первая строка поля, где $n = m = 0$, представляет собой окончание входных данных и обрабатываться не должна.

Выходные данные

Для каждого поля выведите сообщение `Field #x:`, единственное на строке, где x обозначает номер текущего поля, начиная с 1. Следующие n строк должны содержать поле с символами «.» , замененными на число мин в соседних клеточках. Между выводами полей должна быть пустая строка.

Пример входных данных

```
4 4
* . . .
. . . .
. * . .
. . . .
3 5
** . . .
. . . .
. * . .
0 0
```

Соответствующие выходные данные

```
Field #1:
*100
2210
1*10
1110

Field #2:
**100
33200
1*100
```

Спроектирована UML диаграмма решения задачи (Сапёр), в которой входные данные читаются из файла.

UML диаграмма (Сапёр):

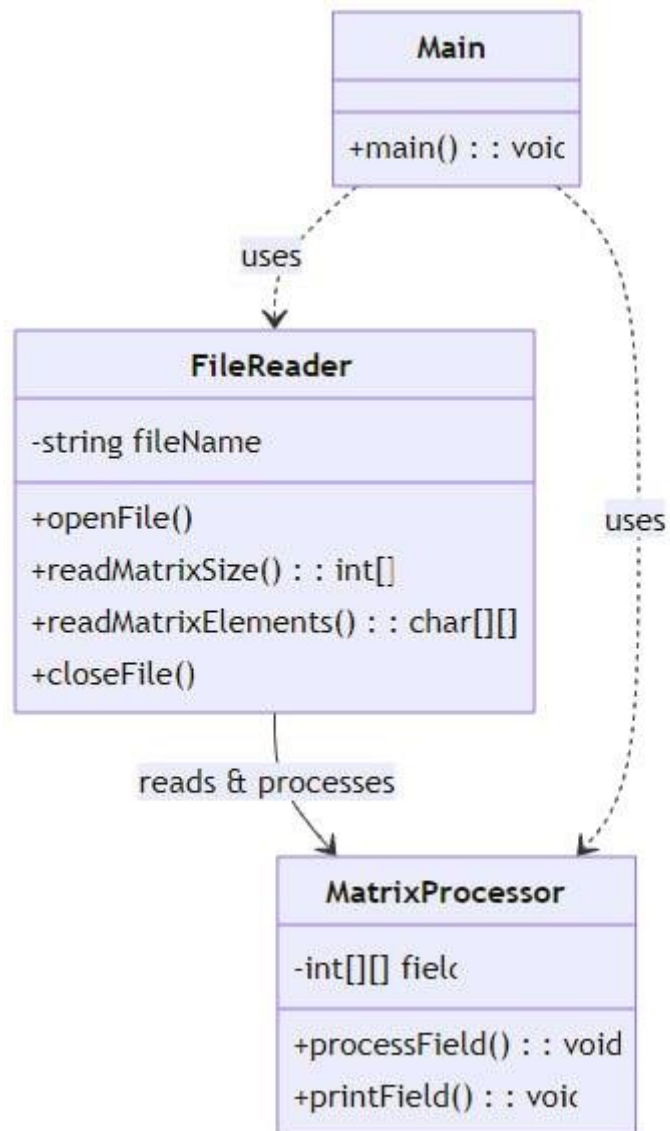


Рисунок 1 – UML диаграмма программы «Сапёр»

Реализация программы подразумевает использование модулей для работы с файловой системой и методов обработки данных.

Код программы. «Сапер» написанный на языке программирования C++:

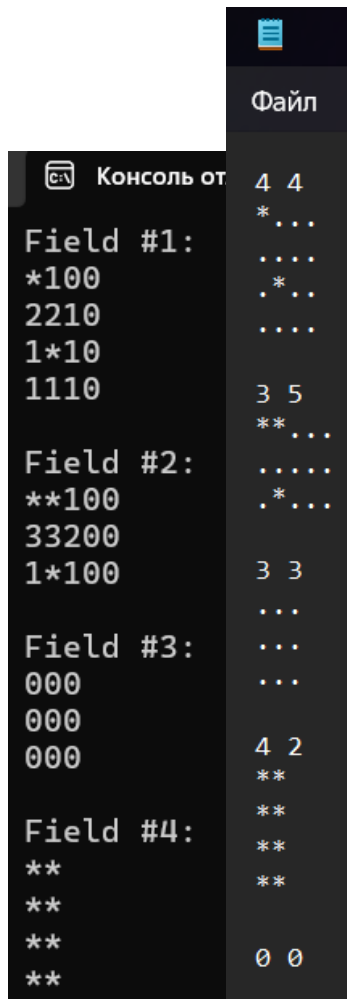
```
#include<stdio.h>
#include<fstream>
#include<string>
#include<iostream>
using namespace std;

int main()
{
    setlocale(LC_ALL, "RUS");

    int n, m, counter=0;
    char cn, cm;
    ifstream file("input.txt");//Открыли файл
    while (true)
    {
        file >> cn;file>> cm;//Читаем из файла размеры матрицы
        n = cn - '0'; m = cm - '0';//Переводим их в числа
        if (n ==0)break;
        counter++;
        if (counter > 1)cout << '\n';
        int field[102][102] = { 0 };
        for(int i=1;i<n+1;i++)//Основная логика
            for (int j = 1;j < m + 1; j++)
            {
                char elem;
                file >> elem;
                if (elem == '*')
                {
                    field[i][j] = -1;
                    if (field[i - 1][j] != -1)field[i - 1][j]++;//Верх
                    if (field[i - 1][j - 1] != -1)field[i - 1][j -
1]++;//Левый верх
                    if (field[i - 1][j + 1] != -1)field[i - 1][j +
1]++;//Правый верх
                    if (field[i][j - 1] != -1)field[i][j - 1]++;//Лево
                    if (field[i + 1][j] != -1)field[i + 1][j]++;//Низ
                    if (field[i + 1][j - 1] != -1)field[i + 1][j -
1]++;//Левый низ
                    if (field[i + 1][j + 1] != -1)field[i + 1][j +
1]++;//Правый низ
                    if (field[i][j + 1] != -1)field[i][j + 1]++;//Право
                }
            }
        cout << "Field #" << counter << ":\n";//Печать готовой новой матрицы
        for (int i = 1; i < n + 1; i++)
        {
            for (int j = 1; j < m + 1; j++)
            {
                if (field[i][j] == -1)cout << '*';//Замена мишеней на '*'
                else cout << field[i][j];
            }
            cout << "\n";
        }
        file.close();//Закрываем файл
        return 0;
    }
}
```

После успешно выполненных операция над исходными данными пользователю выводятся преобразованные матрицы.

Вывод результатов программы Сапёр:



```
Файл
Консоль от 4 4
* ...
Field #1: ...
*100 ...
2210 ...
1*10 ...
1110 3 5
** ...
Field #2: ...
**100 . * ...
33200
1*100 3 3
...
Field #3: ...
000 ...
000
000 4 2
**
**
Field #4: **
**
**
** 0 0
```

Рисунок 2 – Вывод тестовых сценариев программы «Сапёр»

Задача 2.6 (Jolly Jumpers)

PC/UVa IDs: 110108/10142

Последовательность $n > 0$ целых чисел называется jolly jumper, если абсолютные значения разностей последовательных элементов принимают все возможные значения от 1 до $n - 1$. К примеру,

1 4 2 3

это jolly jumper, потому что абсолютные разности равны 3, 2 и 1 соответственно. Определение подразумевает, что любая последовательность из одного числа - это jolly jumper. Напишите программу, которая определяет, является ли каждая из введенных последовательностей jolly jumper.

Входные данные

Каждая строка входных данных содержит число $n < 3000$, за которым следуют n целых чисел, представляющих собой последовательность.

Выходные данные

Для каждой строки входных данных выведите строку, говорящую "Jolly" или "Not jolly".

Пример входных данных

4 1 4 2 3
5 1 4 2 -1 6

Спроектирована UML диаграмма решения задачи (Jolly Jumpers), в которой входные данные читаются из файла, затем обрабатываются входные данные проводится анализ и выводится результат проверки последовательности.

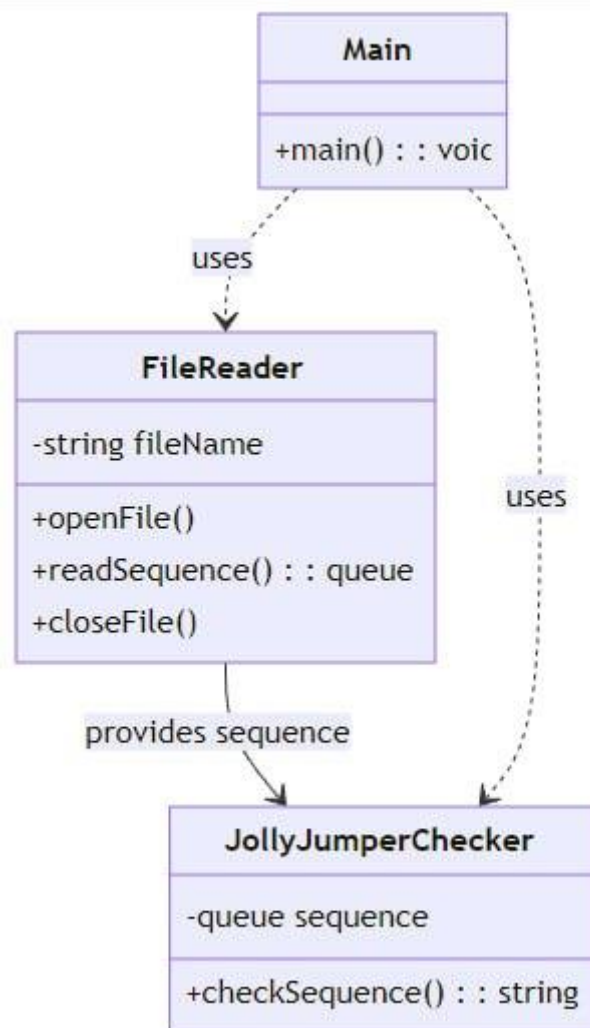


Рисунок 3 - UML диаграмма программы «Jolly Jumpers»

Реализация программы подразумевает использование модулей для работы с файловой системой и методов работы с очередью.

Код программы, написанный на языке программирования C++:

```
#include<stdio.h>
#include<fstream>
#include<string>
```

```

#include<queue>
#include<math.h>
#include<iostream>
using namespace std;

int main()
{
    setlocale(LC_ALL, "RUS");

    ifstream file("input.txt");//Открыли файл
    queue<string> queue;//Очередь для хранения данных
    string symbol;
    int size, intsymbol, n, memory=0, check=0;
    while (!file.eof())//Заполняем очередь из файла
    {
        file >> symbol;//Считали в переменную
        queue.push(symbol);//И добавили в очередь
    }

    while (!queue.empty())//Пока очередь не опустошена
    {
        n = stoi(queue.front());//Получили n
        queue.pop();//Удаление этого числа из очереди
        cout << n << " ";
        for (int i = 0; i < n; i++)//Работа только с нужной последовательностью
        {
            intsymbol = stoi(queue.front());//Переводим строку в число
            memory = abs(memory - intsymbol);//Вычисляем разницу соседних
чисел

            if (memory == (n - i))check++;//Проверка на jolly jumpers
            memory = intsymbol;//Запоминаем число для будущей итерации
            cout << intsymbol << " ";
            queue.pop();//Удаляем число из очереди
        }
        if (check == (n - 1))cout << "Jolly" << endl;
        else cout << "Not Jolly!" << endl;
        check = 0;//Сброс запоминающих переменных
        memory = 0;
    }
    file.close();//Закрыли файл
    return 0;
}

```

После успешно выполненных операций над исходными данными пользователю выводится результат проверки последовательности .

Вывод результатов программы:

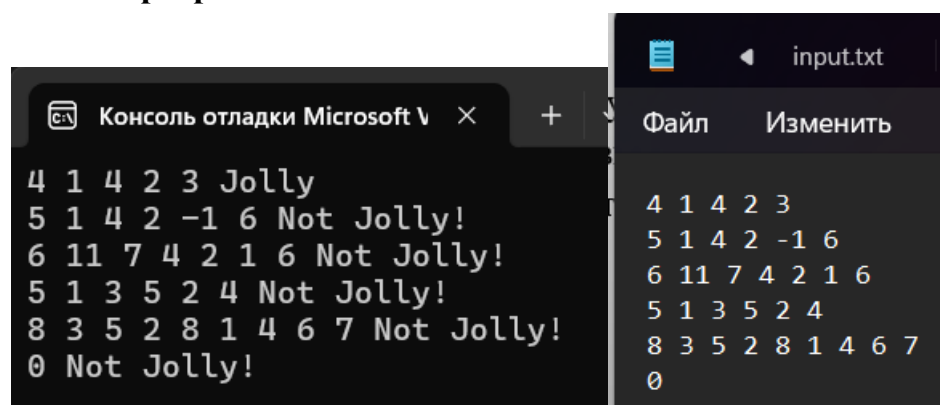


Рисунок 4 – Вывод программы «Jolly Jumpers»

Вывод

В ходе работы были написаны программы на языке программирования C++. Программы решили поставленные задачи («Сапёр», «Jolly Jumpers») и прошли проверку при стандартном тестировании, а также при тестировании с некорректными (по условию задачи) входными данными.